

Компьютерный практикум по статистическому анализу данных

Лабораторная работа № 4. Линейная алгебра

Ведьмина Александра Сергеевна

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
3.1	Базовые операции линейной алгебры	8
3.2	Факторизации и диагонализация	8
3.3	Классификация систем линейных уравнений	9
3.4	Линейная модель экономики	9
3.5	Воспроизводимость и измерение времени	10
4	Выполнение лабораторной работы	11
4.1	Подготовка окружения	11
4.2	Поэлементные операции	13
4.3	Основные матричные операции	15
4.4	Нормы, расстояния и повороты	16
4.5	Матричное и скалярное произведения	17
4.6	Факторизации и специальные структуры	18
4.7	Рациональная линейная алгебра	19
5	Самостоятельные задания	21
5.1	Произведение векторов	21
5.2	Решение СЛАУ	22
5.3	Диагонализация матриц	24
5.4	Степень и корни матриц	26
5.5	Собственные значения и эффективность	27
5.6	Продуктивность линейных моделей экономики	29
6	Воспроизводимые материалы	33
6.1	Раздельный запуск заданий	33
6.2	Генерация Julia, Notebook и Quarto	33
6.3	Выполнение в Jupyter Lab	36
6.4	Quarto и итоговая проверка	36
7	Обсуждение результатов	38
8	Заключение	39

Список литературы	40
А Ключевые вспомогательные функции	41
А.1 Приведение к ступенчатому виду	41
А.2 Регистрация отдельных заданий	42

Список иллюстраций

4.3	Подготовка программного окружения	11
4.6	Создание и настройка проекта	12
4.8	Проверка окружения	12
4.9	Поэлементные операции над массивом	14
4.10	След, ранг, определитель, обратная и псевдообратная матрицы	15
4.11	Нормы, расстояние, угол и повороты матрицы	17
4.12	Матричное и скалярное произведения	18
4.13	LU-, QR-, спектральное разложения и специальные структуры	19
4.14	Точное решение и LU-разложение рациональной матрицы	20
5.1	Скалярное и внешнее произведения вектора	21
5.2	Классификация СЛАУ с двумя неизвестными	23
5.3	Классификация СЛАУ с тремя неизвестными	24
5.4	Диагонализация трёх матриц	26
5.5	Степень, квадратные и кубический корни матриц	27
5.6	Собственные значения, структуры и время операций	29
5.7	Проверка продуктивности по определению	30
5.8	Проверка матрицы Леонтьева	31
5.9	Проверка спектрального критерия	32
6.2	Генерация воспроизводимых материалов	35
6.3	Запуск сгенерированного Julia-сценария	35
6.5	Работа с Notebook	36
6.6	Выполнение Notebook и рендер Quarto	37

Список таблиц

4.1	Поэлементные характеристики массива	14
4.2	Сравнение обработки плотных матриц 1000×1000	19
5.1	СЛАУ с двумя неизвестными	23
5.2	СЛАУ с тремя неизвестными	24
5.3	Результаты диагонализации	25
5.4	Матричные степени и корни	27
5.5	Собственные значения заданной матрицы	28
5.6	Эффективность операций для матрицы 5×5	28
5.7	Проверка продуктивности по определению	30
5.8	Критерий неотрицательности $(I - A)^{-1}$	30
5.9	Спектральный критерий продуктивности	31
6.1	Итоговый состав воспроизводимых материалов	37

1 Цель работы

Изучить возможности специализированных средств Julia для выполнения операций линейной алгебры и оценки их эффективности.

2 Задание

В ходе лабораторной работы требовалось:

1. в Jupyter Lab повторить примеры раздела 4.2 методических указаний;
2. выполнить поэлементные операции, транспонирование, вычисление следа, ранга, определителя, обратной и псевдообратной матриц;
3. исследовать нормы, расстояния, угол между векторами и повороты матриц;
4. выполнить матричные произведения, LU-, QR- и спектральное разложения;
5. сравнить вычисления для плотных и структурированных матриц;
6. выполнить скалярное и внешнее произведения заданного вектора;
7. решить и классифицировать СЛАУ с двумя и тремя неизвестными;
8. диагонализировать матрицы, вычислить степень и матричные корни;
9. найти собственные значения матрицы 5×5 и оценить время операций;
10. проверить продуктивность линейных моделей экономики тремя критериями;
11. оформить вычисления как воспроизводимый проект DrWatson и получить Julia-, Jupyter- и Quarto-представления.

3 Теоретическое введение

3.1 Базовые операции линейной алгебры

Стандартная библиотека `LinearAlgebra` предоставляет матричное умножение, транспонирование, нормы, определители, решения линейных систем, факторизации и спектральные разложения [5]. Для вектора $x = (x_1, \dots, x_n)^T$ евклидова норма определяется формулой

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}.$$

Расстояние между векторами равно $\|x - y\|_2$, а угол между ними вычисляется как

$$\varphi = \arccos \frac{x^T y}{\|x\|_2 \|y\|_2}.$$

Средние значения элементов и срезов массивов вычисляются средствами библиотеки `Statistics` [6].

3.2 Факторизации и диагонализация

LU-разложение представляет матрицу в форме $PA = LU$, где P — матрица перестановок, L — нижняя, а U — верхняя треугольная матрица. QR-разложение имеет вид $A = QR$, где $Q^T Q = I$, а R верхнетреугольна.

Для вещественной симметричной матрицы спектральное разложение записывается как

$$A = PDP^T,$$

где столбцы P являются ортонормированными собственными векторами, а D — диагональная матрица собственных значений. Функция от симметричной матрицы вычисляется спектральным методом:

$$f(A) = Pf(D)P^T.$$

3.3 Классификация систем линейных уравнений

Для системы $Ax = b$ сравниваются ранги матрицы коэффициентов и расширенной матрицы:

- если $\text{rank } A < \text{rank}[A \mid b]$, система несовместна;
- если ранги равны числу неизвестных, решение единственно;
- если ранги равны, но меньше числа неизвестных, решений бесконечно много.

В работе применялся метод Гаусса–Жордана над `Rational{BigInt}`, поэтому ответы для СЛАУ получены без округления.

3.4 Линейная модель экономики

Модель Леонтьева задаётся равенством

$$x - Ax = y, \quad x = (I - A)^{-1}y.$$

Матрица A продуктивна, если для любого $y \geq 0$ решение удовлетворяет $x \geq 0$. Использовались три эквивалентных подхода:

1. непосредственный поиск решения или контрпримера;

2. проверка неотрицательности всех элементов $(I - A)^{-1}$;
3. проверка спектрального радиуса $\rho(A) < 1$.

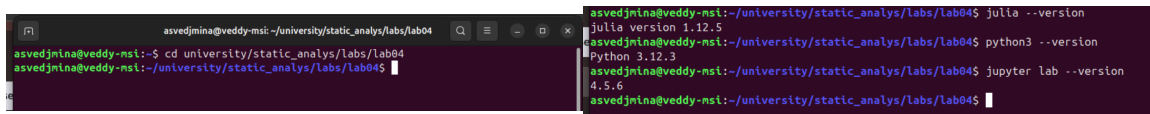
3.5 Воспроизводимость и измерение времени

Структура проекта сформирована пакетом `DrWatson`, который стандартизирует пути к исходникам, сценариям, данным и результатам [2]. Из единого комментированного файла пакет `Literate` создаёт Julia-скрипт, Notebook и Quarto-документ [3; 4]. Для измерений применялся `BenchmarkTools`, уменьшающий влияние компиляции и случайных фоновых задержек [1].

4 Выполнение лабораторной работы

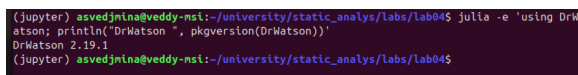
4.1 Подготовка окружения

Работа выполнялась в каталоге `labs/lab04`. Были проверены Julia 1.12.5, Python 3.12.3, Jupyter Lab 4.5.6 и DrWatson 2.19.1 (Рисунок 4.3).



(a) Переход в каталог лабораторной

(a) Версии Julia, Python и Jupyter Lab



(a) Проверка версии DrWatson

Рисунок 4.3: Подготовка программного окружения

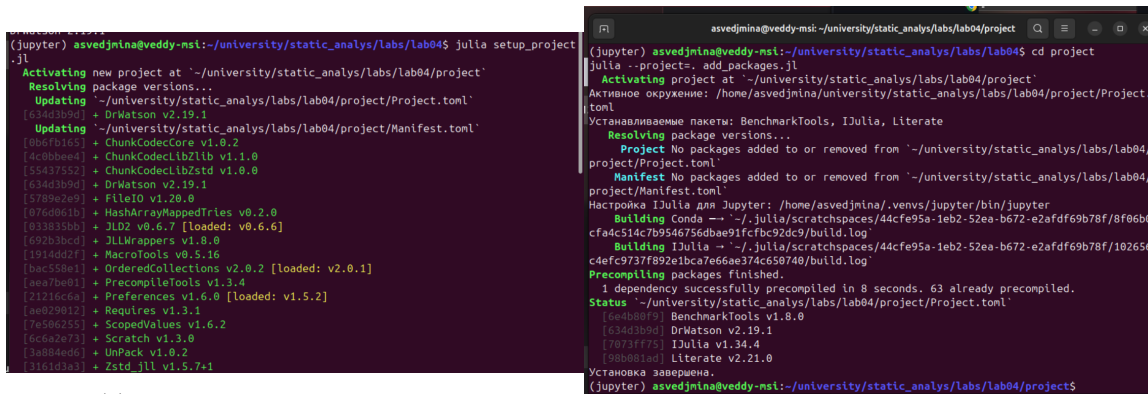
Проект создавался функцией `initialize_project` без инициализации Git:

```
initialize_project(  
    PROJECT_DIRECTORY;  
    authors=["XXXXXXXXXX XXXXXXXXXXXX XXXXXXXXXXXX"],  
    git=false,
```

placeholder=true,

)

Установка пакетов и полученная структура окружения показаны на Рисунк 4.6.

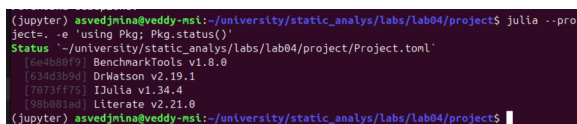


```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia setup_project.jl
Activating new project at `~/university/static_analys/labs/lab04/project`
Resolving package versions...
Updating `~/university/static_analys/labs/lab04/project/Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `~/university/static_analys/labs/lab04/project/Manifest.toml`
[806f8165] + ChunkCodeCore v1.0.2
[4c0bbee4] + ChunkCodeLibZlib v1.1.0
[55437552] + ChunkCodeLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FFI10 v1.20.0
[0760661b] + HashArrayMappedTries v0.2.0
[833833ab] + LLVM v0.6.7 [loaded: v0.6.6]
[692b3bcd] + JLLWrappers v1.8.0
[1914dd2f] + MacroTools v0.5.16
[bac558e1] + OrderedCollections v2.0.2 [loaded: v2.0.1]
[aea7be01] + PrecompileTools v1.3.4
[21216c6a] + Preferences v1.6.0 [loaded: v1.5.2]
[ae029012] + Requires v1.3.1
[7e906255] + ScopedValues v1.6.2
[4c6a2e73] + Scratch v1.3.0
[3a804e05] + UnPack v1.0.2
[316143a3] + Zstd_jll v1.5.7+1

(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$ julia -p project
julia -p project: add_packages.jl
Activating project at `~/university/static_analys/labs/lab04/project`
Активное окружение: /home/asvedjmina/university/static_analys/labs/lab04/project/Project.toml
Устанавливаемые пакеты: BenchmarkTools, IJulia, Literate
Resolving package versions...
Project No packages added to or removed from `~/university/static_analys/labs/lab04/project/Project.toml`
Manifest No packages added to or removed from `~/university/static_analys/labs/lab04/project/Manifest.toml`
Настройка IJulia для Jupyter: /home/asvedjmina/.venvs/jupyter/bin/jupyter
Building Conda → `~/julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/8f66b0cfa4c514c7b9546756dbae91fc92dc9/build.log`
Building IJulia → `~/julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/102656c4efc937f892e1bca7e66ae374c650740/build.log`
Precompiling packages finished.
1 dependency successfully precompiled in 8 seconds. 63 already precompiled.
Status `~/university/static_analys/labs/lab04/project/Project.toml`
[6e4b88f9] BenchmarkTools v1.8.0
[634d3b9d] DrWatson v2.19.1
[7073ff75] IJulia v1.34.4
[98b081ad] Literate v2.21.0
Установка завершена.
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$
```

(a) Создание проекта DrWatson

(a) Установка зависимостей проекта



```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$ julia --project
julia --project: using Pkg; Pkg.status()
Status `~/university/static_analys/labs/lab04/project/Project.toml`
[6e4b88f9] BenchmarkTools v1.8.0
[634d3b9d] DrWatson v2.19.1
[7073ff75] IJulia v1.34.4
[98b081ad] Literate v2.21.0
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$
```

(a) Состояние окружения Julia

Рисунок 4.6: Создание и настройка проекта

Сценарий проверки подтвердил доступность DrWatson, BenchmarkTools, IJulia и Literate. После этого была проверена интерактивная Julia (Рисунок 4.8).



```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$ julia --project
julia --project: scripts/test_setup.jl
Проверка окружения лабораторной работы № 4
=====
Версия Julia: 1.12.5
Активный проект: /home/asvedjmina/university/static_analys/labs/lab04/project/Project.toml
Корень проекта: /home/asvedjmina/university/static_analys/labs/lab04/project
Каталог scripts: /home/asvedjmina/university/static_analys/labs/lab04/project/scripts
Каталог src: /home/asvedjmina/university/static_analys/labs/lab04/project/src

Проверка пакетов:
[OK] DrWatson
[OK] BenchmarkTools
[OK] IJulia
[OK] Literate

Проверка завершена успешно: окружение готово.
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$

(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04/project$ julia
julia> | Documentation: https://docs.julialang.org
Type "?" for help, "]" for Pkg help.
Version 1.12.5 (2026-02-09)
Official https://julialang.org release
```

(a) Интерактивный сеанс Julia

(a) Проверка путей и пакетов проекта

Рисунок 4.8: Проверка окружения

Основной сценарий активирует окружение независимо от текущего каталога:

```
using DrWatson
@quickactivate "project"

using BenchmarkTools
using LinearAlgebra
using Random
using Statistics
```

4.2 Поэлементные операции

Для матрицы 4×3 вычислены сумма, произведение и среднее всех элементов, а также соответствующие значения по строкам и столбцам:

```
a = rand(1:20, (4, 3))

sum(a)
sum(a; dims=1)
sum(a; dims=2)

prod(a)
prod(a; dims=1)
prod(a; dims=2)

mean(a)
mean(a; dims=1)
mean(a; dims=2)
```

Таблица 4.1: Поэлементные характеристики массива

Показатель	Всё	По столбцам	По строкам
Сумма	137	[45 47 45]	[30; 55; 38; 14]
Произведение	762753024000	[11520 6270 10560]	[968; 6080; 1800; 72]
Среднее	11,4167	[11,25 11,75 11,25]	[10; 18,333; 12,667; 4,667]

Полученный вывод приведён на Рисунок 4.9.

```
(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.2.1
=====
4.2.1. Поэлементные операции
=====
a =
4x3 Matrix{Int64}:
 8 11 11
16 19 20
15 15 8
 6 2 6
sum(a) = 137
Суммы по столбцам:
1x3 Matrix{Int64}:
 45 47 45
Суммы по строкам:
4x1 Matrix{Int64}:
 30
 55
 38
 14
prod(a) = 762753024000
Произведения по столбцам:
1x3 Matrix{Int64}:
11520 6270 10560
Произведения по строкам:
4x1 Matrix{Int64}:
 968
6080
1800
 72
mean(a) = 11.416666666666666
Средние по столбцам:
1x3 Matrix{Float64}:
11.25 11.75 11.25
Средние по строкам:
4x1 Matrix{Float64}:
10.0
18.333333333333332
12.666666666666666
 4.6666666666666667
```

Рисунок 4.9: Поэлементные операции над массивом

4.3 Основные матричные операции

Для квадратной матрицы вычислены транспонированная и обратная матрицы, след, диагональ, ранг и определитель. Для прямоугольной матрицы построена псевдообратная:

```
transpose(b)
tr(b)
diag(b)
rank(b)
inv(b)
det(b)
pinv(a)
```

Для зафиксированного генератором примера получено $\text{tr } b = 49$, $\text{rank } b = 4$ и $\det b \approx 11228$ (Рисунок 4.10).

```
(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.2.2
=====
4.2.2. Основные матричные операции
=====
b =
4×4 Matrix{Int64}:
 8  11  11  12
16  19  20   4
15  15   8  13
 6   2   6  14
transpose(b) =
4×4 transpose{::Matrix{Int64}} with eltype Int64:
 8  16  15   6
11  19  15   2
11  20   8   6
12   4  13  14
tr(b) = 49
diag(b) = [8, 19, 8, 14]
rank(b) = 4
inv(b) =
4×4 Matrix{Float64}:
-0.243855   0.067866   0.0771286   0.118009
 0.171714  -0.0602066   0.0391877  -0.16637
 0.0174564   0.0623441  -0.0997506   0.0598504
 0.0724973  -0.0472034   0.0040969   0.0189704
det(b) = 11227.999999999998
pinv(a) =
3×4 Matrix{Float64}:
-0.0218934  -0.0777537  -0.00333728   0.113456
 0.0507021  -0.0441953   0.0454839  -0.0846326
 0.00676641  0.185411   -0.0200293  -0.0595931
```

Рисунок 4.10: След, ранг, определитель, обратная и псевдообратная матрицы

4.4 Нормы, расстояния и повороты

```
X = [2, 4, -5]
Y = [1, -1, 3]

distance = norm(X - Y)
angle = acos(
    clamp(
        dot(X, Y) / (norm(X) * norm(Y)),
        -1,
        1,
    ),
)

opnorm(d)
opnorm(d, 1)
rot180(d)
reverse(d; dims=1)
reverse(d; dims=2)
```

Получены $\|X\|_2 \approx 6,7082$, расстояние $\|X - Y\|_2 \approx 9,4868$ и угол $\varphi \approx 2,4404$ рад. Повороты и перестановки элементов матрицы показаны на Рисунок 4.11.

```

(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.2.3
=====
4.2.3. Нормы, расстояние, угол и повороты
=====
norm(X) = 6.708203932499369
norm(X, 1) = 11.0
norm(X - Y) = 9.486832980505138
Проверка расстояния = 9.486832980505138
Угол X и Y, рад = 2.4404307889469252
d =
3x3 Matrix{Int64}:
 5 -4  2
-1  2  3
-2  1  0
орnorm(d) = 7.147682841795258
орnorm(d, 1) = 8.0
rot180(d) =
3x3 Matrix{Int64}:
 0  1 -2
 3  2 -1
 2 -4  5
reverse(d, dims=1) =
3x3 Matrix{Int64}:
-2  1  0
-1  2  3
 5 -4  2
reverse(d, dims=2) =
3x3 Matrix{Int64}:
 2 -4  5
 3  2 -1
 0  1 -2

```

Рисунок 4.11: Нормы, расстояние, угол и повороты матрицы

4.5 Матричное и скалярное произведения

Оператор `*` применён к согласованным матрицам размеров 2×3 и 3×4 . Единичная матрица сформирована типом `I`, а скалярное произведение проверено двумя способами:

```

A_product * B_product
Matrix{Int}(I, 3, 3)
dot(X, Y)
X'Y

```

Оба способа дали $X^T Y = -17$ (Рисунок 4.12).

```
(jupyter) asvedjmina@veddy-nst:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.2.4
=====
4.2.4. Умножение матриц и векторов
=====
A * B =
2×4 Matrix{Int64}:
 76  90  82 132
127 122 124 126
Единичная матрица =
3×3 Matrix{Int64}:
 1  0  0
 0  1  0
 0  0  1
dot(X, Y) = -17
X'Y = -17
```

Рисунок 4.12: Матричное и скалярное произведения

4.6 Факторизации и специальные структуры

```
A_lu = lu(A_factor)
```

```
A_lu.P
```

```
A_lu.L
```

```
A_lu.U
```

```
A_lu \ b_factor
```

```
A_qr = qr(A_factor)
```

```
Q = Matrix(A_qr.Q)
```

```
R = A_qr.R
```

```
A_eigen = eigen(
```

```
    Symmetric(A_factor + A_factor'),
```

```
)
```

Проверки $PA \approx LU$, $Q^T Q \approx I$, $QR \approx A$ и $PDP^T \approx A$ дали true. Для матриц большой размерности сравнивались обычная симметричная, зашумлённая и явно объявленная `Symmetric` структуры. Матрица `SymTridiagonal` размера $10^6 \times 10^6$ обработана без перехода к плотному представлению, которому потребовалось бы примерно 7,28 ТиБ памяти (Рисунок 4.13).

```

(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.2.5
=====
4.2.5. Факторизации
=====
Решение A \ b = [1.0, 1.0000000000000007, 1.0]
Матрица перестановок P =
3×3 Matrix{Float64}:
 0.0  1.0  0.0
 1.0  0.0  0.0
 0.0  0.0  1.0
Вектор перестановок p = [2, 1, 3]
L =
3×3 Matrix{Float64}:
 1.0  0.0  0.0
 0.376922  1.0  0.0
 0.704417  0.298553  1.0
U =
3×3 Matrix{Float64}:
 0.941439  0.347112  0.250707
 0.0  -0.0401004  0.454283
 0.0  0.0  0.5892
Решение через LU = [1.0, 1.0000000000000007, 1.0]
det(A) = 0.02224352767599258, det(lu(A)) = 0.02224352767599258
Проверка P*A ≈ L*U: true
Q =
3×3 Matrix{Float64}:
 -0.294481  0.922691 -0.24884
 -0.78128  -0.382396 -0.49333
 -0.550347  0.0491374  0.833489
R =
3×3 Matrix{Float64}:
 -1.205 -0.425889 -0.853577
 0.0 -0.0375886  0.45478
 0.0 0.0 0.491092
Q'Q ≈ I: true
Q*R ≈ A: true
Собственные значения = [-0.4889839819705446, 0.6853608444506039, 3.010405405912214]
Собственные векторы =
3×3 Matrix{Float64}:
 0.776559 -0.286183 0.561298

```

Рисунок 4.13: LU-, QR-, спектральное разложения и специальные структуры

Таблица 4.2: Сравнение обработки плотных матриц 1000×1000

Представление	Время вычисления eigvals, c
Симметричная плотная матрица	0,0654
Зашумлённая матрица общего вида	0,4229
Явная структура Symmetric	0,0656

4.7 Рациональная линейная алгебра

Матрица с рациональными элементами создавалась без перехода к Float64:

```

A_rational =
    Rational{BigInt}.(rand(1:10, 3, 3)) ./ 10
x_rational = fill(BigInt(1), 3)
b_rational = A_rational * x_rational

A_rational \ b_rational
lu(A_rational)

```

Решение точно равно $[1, 1, 1]^T$; LU-разложение также содержит рациональные числа (Рисунок 4.14).

```

(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.2.6
=====
4.2.6. Рациональная линейная алгебра
=====
A_rational =
3×3 Matrix{Rational{BigInt}}:
 2//5  3//10  4//5
 4//5  3//5   1//10
 4//5   1   3//5
Точное решение = Rational{BigInt}[1, 1, 1]
LU для рациональной матрицы =
LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
L factor:
3×3 Matrix{Rational{BigInt}}:
 1  0  0
 1  1  0
 1//2  0  1
U factor:
3×3 Matrix{Rational{BigInt}}:
 4//5  3//5  1//10
 0  2//5  1//2
 0  0  3//4

```

Рисунок 4.14: Точное решение и LU-разложение рациональной матрицы

5 Самостоятельные задания

5.1 Произведение векторов

```
v = [1, 2, 3]
dot_v = dot(v, v)
outer_v = v * v'
```

Получены $v^T v = 14$ и внешнее произведение

$$vv^T = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}.$$

```
(jupyter) asvedjmlna@veddy-msi:~/university/static_analys/labs/Lab04$ julia --project=project project/scripts/run_task.jl 4.4.1
=====
4.4.1. Произведение векторов
=====
v = [1, 2, 3]
dot_v = v ⋅ v = 14
outer_v = v*v' =
3×3 Matrix{Int64}:
 1  2  3
 2  4  6
 3  6  9
```

Рисунок 5.1: Скалярное и внешнее произведения вектора

5.2 Решение СЛАУ

Для точной классификации вычислялась приведённая ступенчатая форма расширенной матрицы. Основной интерфейс функции имеет вид:

```
function solve_exact(A, b)
    augmented = hcat(
        Rational{BigInt}.(A),
        Rational{BigInt}.(b),
    )
    reduced = rref_exact(augmented)

    rank_a = count(
        row -> any(!iszero,
                    reduced[row, 1:size(A, 2)]),
        1:size(A, 1),
    )
    rank_augmented = count(
        row -> any(!iszero, reduced[row, :]),
        1:size(A, 1),
    )
    # 4x4 8x8 4x4 4x4
    # 1 4x4 8x8 8x8 8x8.
end
```

Результаты систем с двумя неизвестными приведены в Таблица 5.1 и на Рисунок 5.2.

Таблица 5.1: СЛАУ с двумя неизвестными

Система	Тип	Решение или параметризация
a	единственное	$(x, y) = (5/2, -1/2)$
b	бесконечно много	$(2, 0) + t(-1, 1)$
c	несовместна	решений нет
d	бесконечно много	$(1, 0) + t(-1, 1)$
e	несовместна	решений нет
f	единственное	$(x, y) = (-1, 3)$

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.2.1
=====
4.4.2. СЛАУ с двумя неизвестными
=====

a: единственное решение
rank(A) = 2, rank([A|b]) = 2
RREF([A|b]) =
2×3 Matrix{Rational{BigInt}}:
 1  0  5//2
 0  1 -1//2
Частное решение: Rational{BigInt}[5//2, -1//2]

b: бесконечно много решений
rank(A) = 1, rank([A|b]) = 1
RREF([A|b]) =
2×3 Matrix{Rational{BigInt}}:
 1  1  2
 0  0  0
Частное решение: Rational{BigInt}[2, 0]
Базис однородных решений: Vector{Rational{BigInt}}[[-1, 1]]
Общий вид:  $x = x_0 + \sum t_i \cdot n_i$ 

c: решений нет
rank(A) = 1, rank([A|b]) = 2
RREF([A|b]) =
2×3 Matrix{Rational{BigInt}}:
 1  1  0
 0  0  1

d: бесконечно много решений
rank(A) = 1, rank([A|b]) = 1
RREF([A|b]) =
3×3 Matrix{Rational{BigInt}}:
 1  1  1
 0  0  0
 0  0  0
Частное решение: Rational{BigInt}[1, 0]
Базис однородных решений: Vector{Rational{BigInt}}[[-1, 1]]
Общий вид:  $x = x_0 + \sum t_i \cdot n_i$ 

e: решений нет
rank(A) = 2, rank([A|b]) = 3
```

Рисунок 5.2: Классификация СЛАУ с двумя неизвестными

Для систем с тремя неизвестными получены результаты из Таблица 5.2 и Рисунок 5.3.

Таблица 5.2: СЛАУ с тремя неизвестными

Система	Тип	Решение или параметризация
a	бесконечно много	$(5/2, -1/2, 0) + t(1/2, -3/2, 1)$
b	единственное	$(-1/2, 5/2, 0)$
c	бесконечно много	$(2, 0, -1) + t(-1, 1, 0)$
d	несовместна	решений нет

```
(jupyter) asvedjina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.2.2
=====
4.4.2. СЛАУ с тремя неизвестными
=====

a: бесконечно много решений
rank(A) = 2, rank([A|b]) = 2
RREF([A|b]) =
2x4 Matrix{Rational{BigInt}}:
 1  0 -1//2  5//2
 0  1  3//2 -1//2
Частное решение: Rational{BigInt}[5//2, -1//2, 0]
Базис однородных решений: Vector{Rational{BigInt}}[[1//2, -3//2, 1]]
Общий вид:  $x = x_0 + \sum t_i \cdot n_i$ 

b: единственное решение
rank(A) = 3, rank([A|b]) = 3
RREF([A|b]) =
3x4 Matrix{Rational{BigInt}}:
 1  0  0 -1//2
 0  1  0  5//2
 0  0  1  0
Частное решение: Rational{BigInt}[-1//2, 5//2, 0]

c: бесконечно много решений
rank(A) = 2, rank([A|b]) = 2
RREF([A|b]) =
3x4 Matrix{Rational{BigInt}}:
 1  1  0  2
 0  0  1 -1
 0  0  0  0
Частное решение: Rational{BigInt}[2, 0, -1]
Базис однородных решений: Vector{Rational{BigInt}}[[-1, 1, 0]]
Общий вид:  $x = x_0 + \sum t_i \cdot n_i$ 

d: решений нет
rank(A) = 2, rank([A|b]) = 3
RREF([A|b]) =
3x4 Matrix{Rational{BigInt}}:
 1  1  0  0
 0  0  1  0
 0  0  0  1
```

Рисунок 5.3: Классификация СЛАУ с тремя неизвестными

5.3 Диагонализация матриц

```

decomposition = eigen(
    Symmetric(Float64.(matrix)),
)
D = Diagonal(decomposition.values)
P = decomposition.vectors
residual = norm(matrix * P - P * D)

```

Таблица 5.3: Результаты диагонализации

Матрица	Собственные значения	Норма невязки
a	−1; 3	0
b	−0,236068; 4,23607	$7,12 \cdot 10^{-16}$
c	−2,14134; 0,515138; 3,6262	$7,18 \cdot 10^{-15}$

```

(jupyter) asvedjnina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.3.1
=====
4.4.3.1. Диагонализация
=====

a) A =
2×2 Matrix{Int64}:
 1 -2
-2  1
D =
2×2 Diagonal{Float64, Vector{Float64}}:
-1.0  .
.    3.0
P (столбцы — собственные векторы) =
2×2 Matrix{Float64}:
-0.707107 -0.707107
-0.707107  0.707107
Проверка A*P ≈ P*D, невязка = 0.0

b) A =
2×2 Matrix{Int64}:
 1 -2
-2  3
D =
2×2 Diagonal{Float64, Vector{Float64}}:
-0.236068 .
.    4.23607
P (столбцы — собственные векторы) =
2×2 Matrix{Float64}:
-0.850651 -0.525731
-0.525731  0.850651
Проверка A*P ≈ P*D, невязка = 7.121076845233165e-16

c) A =
3×3 Matrix{Int64}:
 1 -2  0
-2  1  2
 0  2  0
D =
3×3 Diagonal{Float64, Vector{Float64}}:
-2.14134 . .
.    0.515138 .
. .    3.6262
P (столбцы — собственные векторы) =
3×3 Matrix{Float64}:
-0.515138  0.850651  0.196127
0.850651  0.525731  0.0371355
0.196127  0.0371355  0.962863

```

Рисунок 5.4: Диагонализация трёх матриц

5.4 Степень и корни матриц

Функции от симметричных матриц вычислялись через собственные значения:

```

function matrix_function_symmetric(A, f)
    decomposition = eigen(
        Symmetric(Float64.(A)),
    )
    transformed = f.(decomposition.values)
    return decomposition.vectors *
        Diagonal(transformed) *
        decomposition.vectors'
end

```

end

Таблица 5.4: Матричные степени и корни

Пункт	Операция	Контроль результата
a	$[1 \ -2; -2 \ 1]^{10}$	$[29525 \ -29524; -29524 \ 29525]$
b	квадратный корень $[5 \ -2; -2 \ 5]$	$R^2 \approx A$
c	кубический корень $[1 \ -2; -2 \ 1]$	$R^3 \approx A$
d	главный корень $[1 \ 2; 2 \ 3]$	комплексный, $R^2 \approx A$

Комплексный результат пункта d обусловлен отрицательным собственным значением исходной матрицы (Рисунок 5.5).

```
(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.3.2
=====
4.4.3.2. Степень и корни матриц
=====
a) [1 -2; -2 1]^10 =
2x2 Matrix{Int64}:
 29525  -29524
-29524   29525
b) sqrt([5 -2; -2 5]) =
2x2 Matrix{Float64}:
 2.1889  -0.45685
-0.45685  2.1889
Проверка R^2 ≈ A: true
c) cbrt([1 -2; -2 1]) =
2x2 Matrix{Float64}:
 0.221125  -1.22112
-1.22112  0.221125
Проверка R^3 ≈ A: true
d) Главный комплексный sqrt([1 2; 2 3]) =
2x2 Matrix{ComplexF64}:
 0.568864+0.351578im  0.920442-0.217287im
 0.920442-0.217287im  1.48931+0.134291im
Комплексный результат нужен из-за отрицательного собственного значения.
Проверка R^2 ≈ A: true
```

Рисунок 5.5: Степень, квадратные и кубический корни матриц

5.5 Собственные значения и эффективность

Для заданной симметричной матрицы 5×5 получены собственные значения:

Таблица 5.5: Собственные значения заданной матрицы

Номер	λ_i
1	-128,49323
2	-55,88778
3	42,75217
4	87,16111
5	542,46773

```
eigenvalues = eigvals(Symmetric(large_A))
D = Diagonal(eigenvalues)
L = LowerTriangular(large_A)

time_eigenvalues =
    @belapsed eigvals(Symmetric($large_A))
time_diagonal =
    @belapsed Diagonal($eigenvalues)
time_lower =
    @belapsed LowerTriangular($large_A)
```

Таблица 5.6: Эффективность операций для матрицы 5×5

Операция	Время, с
eigvals	$1,18 \cdot 10^{-6}$
создание Diagonal	$1,71 \cdot 10^{-9}$
создание LowerTriangular	$1,92 \cdot 10^{-9}$

```

(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.3.3
=====
4.4.3.3. Собственные значения и эффективность
=====
Собственные значения A = [-128.49322764802145, -55.88778455305688, 42.75216727931894, 87.16111477514521, 542.4677301466143]
Диагональная матрица собственных значений =
5×5 Diagonal{Float64, Vector{Float64}}:
-128.493      .      .      .      .
.      -55.8878      .      .      .
.      .      42.7522      .      .
.      .      .      87.1611      .
.      .      .      .      542.468
Нижнетреугольная часть A =
5×5 LowerTriangular{Int64, Matrix{Int64}}:
140      .      .      .      .
97 106      .      .      .
74 89 152      .      .
168 131 144 54      .
131 36 71 142 36
Время eigvals: 1.1902999999999999e-6 c
Время создания Diagonal: 1.452e-9 c
Время создания LowerTriangular: 1.9259999999999998e-9 c
=====
Задача 4.4.3.3 выполнена

```

Рисунок 5.6: Собственные значения, структуры и время операций

5.6 Продуктивность линейных моделей экономики

```

function productivity_by_inverse(A; atol=1e-12)
    coefficients = Matrix{Float64}(A)
    leontief_inverse = inv(I - coefficients)
    productive = all(
        value -> value >= -atol,
        leontief_inverse,
    )
    return productive, leontief_inverse
end

```

При проверке по определению для непродуктивных матриц дополнительно строился неотрицательный вектор y , при котором в решении x имеется отрицательный элемент (Рисунок 5.7).

Таблица 5.7: Проверка продуктивности по определению

Пункт	Масштаб матрицы $[1\ 2; 3\ 4]$	Результат
a	1	непродуктивна
b	$1/2$	непродуктивна
c	$1/10$	продуктивна

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.4.1
=====
4.4.4.1. Проверка продуктивности по определению
=====

a) A =
2×2 Matrix{Float64}:
 1.0  2.0
 3.0  4.0
(I-A)^(-1) =
2×2 Matrix{Float64}:
 0.5  -0.333333
-0.5   0.0
Матрица непродуктивна
Контрпример по определению: y = [1.0, 0.0], x = [0.5, -0.5]

b) A =
2×2 Matrix{Float64}:
 0.5  1.0
 1.5  2.0
(I-A)^(-1) =
2×2 Matrix{Float64}:
 0.5  -0.5
-0.75 -0.25
Матрица непродуктивна
Контрпример по определению: y = [1.0, 0.0], x = [0.5, -0.75]

c) A =
2×2 Matrix{Float64}:
 0.1  0.2
 0.3  0.4
(I-A)^(-1) =
2×2 Matrix{Float64}:
 1.25  0.416667
 0.625 1.875
Матрица продуктивна
```

Рисунок 5.7: Проверка продуктивности по определению

Критерий неотрицательности обратной матрицы дал результат из Таблица 5.8 и Рисунок 5.8.

Таблица 5.8: Критерий неотрицательности $(I - A)^{-1}$

Пункт	Матрица	Знак элементов $(I - A)^{-1}$	Результат
a	$[1\ 2; 3\ 1]$	есть отрицательные	непродуктивна
b	$\frac{1}{2}[1\ 2; 3\ 1]$	есть отрицательные	непродуктивна

Пункт	Матрица	Знак элементов $(I - A)^{-1}$	Результат
с	$\frac{1}{10} \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$	все неотрицательны	продуктивна

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.4.2
=====
4.4.4.2. Критерий неотрицательности  $(I-A)^{-1}$ 
=====
a) A =
2×2 Matrix{Float64}:
 1.0  2.0
 3.0  1.0
(I-A)^(-1) =
2×2 Matrix{Float64}:
-0.0  -0.333333
-0.5   0.0
Матрица непродуктивна

b) A =
2×2 Matrix{Float64}:
 0.5  1.0
 1.5  0.5
(I-A)^(-1) =
2×2 Matrix{Float64}:
-0.4  -0.8
-1.2  -0.4
Матрица непродуктивна

c) A =
2×2 Matrix{Float64}:
 0.1  0.2
 0.3  0.1
(I-A)^(-1) =
2×2 Matrix{Float64}:
 1.2  0.266667
 0.4  1.2
Матрица продуктивна
=====
```

Рисунок 5.8: Проверка матрицы Леонтьева

Для спектрального критерия вычислялся максимальный модуль собственного значения:

```
values = eigvals(matrix)
radius = maximum(abs.(values))
productive = radius < 1
```

Таблица 5.9: Спектральный критерий продуктивности

Пункт	Спектральный радиус	Результат
а	3,44949	непродуктивна

Пункт	Спектральный радиус	Результат
b	1,72474	непродуктивна
c	0,344949	продуктивна
d	0,373205	продуктивна

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/run_task.jl 4.4.4.3
=====
4.4.4.3. Спектральный критерий
=====
a)  $\lambda = [-1.4494897427831779, 3.4494897427831783]$ 
Спектральный радиус  $\rho(A) = 3.4494897427831783$ 
Матрица непродуктивна

b)  $\lambda = [-0.7247448713915892, 1.724744871391589]$ 
Спектральный радиус  $\rho(A) = 1.724744871391589$ 
Матрица непродуктивна

c)  $\lambda = [-0.14494897427831785, 0.34494897427831783]$ 
Спектральный радиус  $\rho(A) = 0.34494897427831783$ 
Матрица продуктивна

d)  $\lambda = [0.02679491924311228, 0.1, 0.37320508075688774]$ 
Спектральный радиус  $\rho(A) = 0.37320508075688774$ 
Матрица продуктивна
```

Рисунок 5.9: Проверка спектрального критерия

6 Воспроизводимые материалы

6.1 Раздельный запуск заданий

Все вычислительные фрагменты оформлены как функции и зарегистрированы в списке TASKS. Запускатель передаёт номер задания через переменную окружения:

```
if length(ARGS) != 1
    println(
        "XXXXXXXXXXXXXXXX: " *
        "julia --project=. " *
        "scripts/run_task.jl <XXXX>",
    )
    exit(2)
end

ENV["LAB04_TASK"] = only(ARGS)
include(scriptsdir("lab04_literate.jl"))
```

6.2 Генерация Julia, Notebook и Quarto

Один вызов `generate_outputs.jl` создаёт три представления выбранного задания:

```
script_path = Literate.script(  
    source,  
    output_directory;  
    name=base_name * "_code",  
    preprocess=preprocess,  
    credit=false,  
)  
  
notebook_path = Literate.notebook(  
    source,  
    output_directory;  
    name=base_name * "_notebook",  
    preprocess=preprocess,  
    execute=false,  
    credit=false,  
)  
  
quarto_path = Literate.markdown(  
    source,  
    output_directory;  
    name=base_name * "_quarto",  
    flavor=Literate.QuartoFlavor(),  
    preprocess=preprocess,  
    credit=false,  
)
```

Команда для полного комплекта и пакетная генерация отдельных заданий показаны на Рисунок 6.2.

```
(jupyter) aswed@aswedmaddy-net:~/university/static_analysis/labs/lab04$ julia -project=project project/scripts/generate_outputs.jl all
[Info] generating plain script file from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_1/lab04_a_2_1_code.jl"
[Info] generating notebook from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_1/lab04_a_2_1_notebook.ipynb"
[Info] generating markdown page from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_1/lab04_a_2_1 Quarto.qmd"
Задача: 4.2.1
Julia-схемта: /home/aswed/julia/university/static_analysis/labs/lab04/project/generated/A_2_1/lab04_a_2_1_code.jl
Notebook: /home/aswed/julia/university/static_analysis/labs/lab04/project/generated/A_2_1/lab04_a_2_1_notebook.ipynb
Quarto: /home/aswed/julia/university/static_analysis/labs/lab04/project/generated/A_2_1/lab04_a_2_1_Quarto.qmd

Занесу схемта:
julia --project=. generated/A_2_1/lab04_a_2_1-code.jl

Рендеру Quarto:
quarto render generated/A_2_1/lab04_a_2_1-Quarto.qmd --to-html
[Info] generating plain script file from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_2/lab04_a_2_2_code.jl"
[Info] generating notebook from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_2/lab04_a_2_2_notebook.ipynb"
[Info] generating markdown page from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_2/lab04_a_2_2_Quarto.qmd"
Задача: 4.2.2
Julia-схемта: /home/aswed/julia/university/static_analysis/labs/lab04/project/generated/A_2_2/lab04_a_2_2_code.jl
Notebook: /home/aswed/julia/university/static_analysis/labs/lab04/project/generated/A_2_2/lab04_a_2_2_notebook.ipynb
Quarto: /home/aswed/julia/university/static_analysis/labs/lab04/project/generated/A_2_2/lab04_a_2_2-Quarto.qmd

Занесу схемта:
julia --project=. generated/A_2_2/lab04_a_2_2-code.jl

Рендеру Quarto:
quarto render generated/A_2_2/lab04_a_2_2-Quarto.qmd --to-html
[Info] generating plain script file from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_3/lab04_a_2_3_code.jl"
[Info] generating notebook from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_3/lab04_a_2_3_notebook.ipynb"
[Info] generating markdown page from "~/university/static_analysis/labs/lab04/project/scripts/lab04_iterate.jl"
[Info] writing result to "~/university/static_analysis/labs/lab04/project/generated/A_2_3/lab04_a_2_3-Quarto.qmd"
[Home]
```

(а) Пакетная генерация отдельных заданий

Рисунок 6.2: Генерация воспроизводимых материалов

Сгенерированный Julia-файл запускается как обычный сценарий. На Рисунок 6.3 показан отдельный запуск задания 4.4.3.2.

```

=====
(jupyter) asvedjmlna@veddy-nsi:~/university/static_analys/labs/lab04$ julia --project=project project/generated/4_4_3_2/lab04_4_4_3_2_code.jl

=====
4.4.3.2. Степень и корни матриц
=====
a) [1 -2; -2 1]^10 =
2x2 Matrix{Int64}:
 29525  -29524
-29524  29525
b) sqrt([5 -2; -2 5]) =
2x2 Matrix{Float64}:
 2.1889  -0.45685
-0.45685  2.1889
Проверка R^2 ≈ A: true
c) cbrt([1 -2; -2 1]) =
2x2 Matrix{Float64}:
 0.221125  -1.22112
-1.22112  0.221125
Проверка R^3 ≈ A: true
d) Главный комплексный sqrt([1 2; 2 3]) =
2x2 Matrix{ComplexF64}:
 0.568864+0.351578im  0.920442-0.217287im
 0.920442-0.217287im  1.48931+0.134291im
Комплексный результат нужен из-за отрицательного собственного значения.
Проверка R^2 ≈ A: true

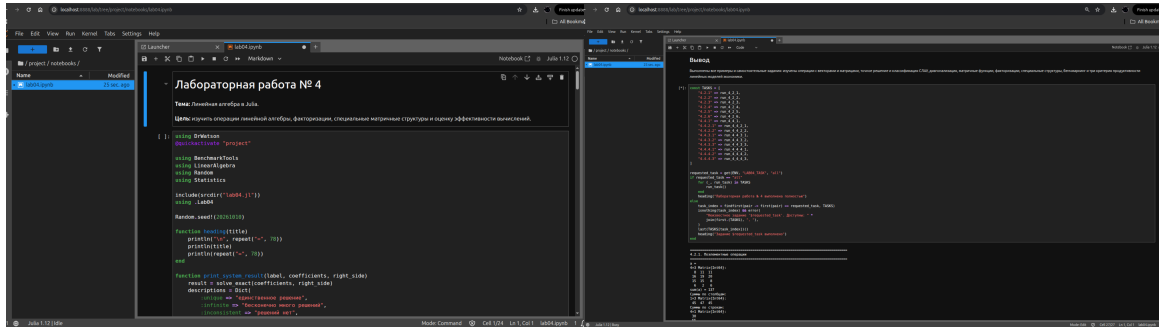
=====
Задание 4.4.3.2 выполнено
=====
(jupyter) asvedjmlna@veddy-nsi:~/university/static_analys/labs/lab04$

```

Рисунок 6.3: Запуск сгенерированного Julia-сценария

6.3 Выполнение в Jupyter Lab

Основной Notebook содержит пояснения, определения функций и результаты всех заданий. На Рисунок 6.5 показаны исходные ячейки и итоговый вывод.



(а) Исходный код лабораторной в Jupyter Lab

(а) Результаты выполнения Notebook

Рисунок 6.5: Работа с Notebook

6.4 Quarto и итоговая проверка

Quarto запустил ядро `julia-1.12`, выполнил все ячейки и создал HTML. В финальной проверке подготовлены 16 комплектов: 15 отдельных заданий и `all`. Каждый комплект содержит Julia-сценарий, выполненный Notebook, QMD и HTML.

```
(jupyter) asvedjmina@veddy-mst:~/university/static_analys/labs/lab04$ julia --project=project project/scripts/generate_outputs.jl 4.4.1
quarto render project/generated/4_4_1/lab04_4_4_1_quarto.qmd --to html
[ Info: generating plain script file from "~/university/static_analys/labs/lab04/project/scripts/lab04_lliterate.jl"
[ Info: writing result to "~/university/static_analys/labs/lab04/project/generated/4_4_1/lab04_4_4_1_code.jl"
[ Info: generating notebook from "~/university/static_analys/labs/lab04/project/scripts/lab04_lliterate.jl"
[ Info: writing result to "~/university/static_analys/labs/lab04/project/generated/4_4_1/lab04_4_4_1_notebook.ipynb"
[ Info: generating markdown page from "~/university/static_analys/labs/lab04/project/scripts/lab04_lliterate.jl"
[ Info: writing result to "~/university/static_analys/labs/lab04/project/generated/4_4_1/lab04_4_4_1_quarto.qmd"
Задание: 4.4.1
Julia-скрипт: /home/asvedjmina/university/static_analys/labs/lab04/project/generated/4_4_1/lab04_4_4_1_code.jl
Notebook: /home/asvedjmina/university/static_analys/labs/lab04/project/generated/4_4_1/lab04_4_4_1_notebook.ipynb
Quarto: /home/asvedjmina/university/static_analys/labs/lab04/project/generated/4_4_1/lab04_4_4_1_quarto.qmd

Запуск скрипта:
julia --project=. generated/4_4_1/lab04_4_4_1_code.jl

Рендер Quarto:
quarto render generated/4_4_1/lab04_4_4_1_quarto.qmd --to html

Starting julia-1.12 kernel...Done

Executing 'lab04_4_4_1_quarto.ipynb'
Cell 1/13: "...Done
Cell 2/13: "...Done
Cell 3/13: "...Done
Cell 4/13: "...Done
Cell 5/13: "...Done
Cell 6/13: "...Done
Cell 7/13: "...Done
Cell 8/13: "...Done
Cell 9/13: "...Done
Cell 10/13: "...Done
Cell 11/13: "...Done
Cell 12/13: "...Done
Cell 13/13: "...Done

pandoc
to: html
output-file: lab04_4_4_1_quarto.html
standalone: true
section-divs: true
html-math-method: mathjax
wrap: none
default-image-extension: png
toc: true
```

Рисунок 6.6: Выполнение Notebook и рендер Quarto

Таблица 6.1: Итоговый состав воспроизводимых материалов

Артефакт	Количество	Проверка
Julia-сценарий	16	создан
Jupyter Notebook по заданию	16	выполнен без ошибок
Основной Notebook	1	выполнен без ошибок
Quarto QMD	16	создан
Quarto HTML	16	отрендерен
Автоматические тесты	14	пройдены

7 Обсуждение результатов

Результаты подтверждают корректность реализованных операций. Невязки диагонализации имеют порядок 10^{-15} , что соответствует погрешности вычислений с `Float64`. Точные рациональные вычисления исключили округление при классификации СЛАУ.

Явное указание структуры `Symmetric` ускорило вычисление собственных значений зашумлённой матрицы примерно в 6,4 раза. Тип `SymTridiagonal` позволил работать с матрицей порядка 10^6 , не выделяя память под 10^{12} элементов.

Три способа проверки продуктивности согласуются: для матриц с обратной матрица Леонтьева неотрицательна и спектральный радиус меньше единицы. Для матриц `a` и `b` обнаружены отрицательные компоненты решения либо $\rho(A) > 1$.

Единый `literate`-источник уменьшает риск расхождения кода между терминалом, Notebook и Quarto. Автоматическое выполнение всех 17 Notebook и рендер всех 16 QMD завершились без ошибок.

8 Заключение

В результате работы:

1. изучены основные операции `LinearAlgebra` и `Statistics`;
2. выполнены LU-, QR- и спектральное разложения;
3. исследована эффективность плотных и структурированных матриц;
4. точно решены и классифицированы десять СЛАУ;
5. выполнены диагонализация, матричные степени и корни;
6. найдены собственные значения и измерено время операций;
7. проверена продуктивность экономических моделей тремя критериями;
8. создан воспроизводимый проект `DrWatson`;
9. сформированы и проверены Julia-, Jupyter- и Quarto-материалы.

Цель лабораторной работы достигнута.

Список литературы

1. *BenchmarkTools contributors*. BenchmarkTools.jl Manual. — URL: <https://juliaci.github.io/BenchmarkTools.jl/stable/> (visited on 10/10/2026).
2. DrWatson: the perfect sidekick for your scientific inquiries / G. Datseris [et al.] // Journal of Open Source Software. — 2020. — Vol. 5, no. 54. — P. 2673. — DOI: 10.21105/joss.02673.
3. *Ekre F.* Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/> (visited on 09/08/2026).
4. *Posit Software, PBC.* Quarto Documentation. — URL: <https://quarto.org/docs/> (visited on 09/08/2026).
5. *The Julia Project.* Linear Algebra — Julia Documentation. — URL: <https://docs.julialang.org/en/v1/stdlib/LinearAlgebra/> (visited on 10/10/2026).
6. *The Julia Project.* Statistics — Julia Documentation. — URL: <https://docs.julialang.org/en/v1/stdlib/Statistics/> (visited on 10/10/2026).

А Ключевые вспомогательные функции

А.1 Приведение к ступенчатому виду

```
function rref_exact(matrix)
    result = Rational{BigInt}.(matrix)
    rows, columns = size(result)
    pivot_row = 1

    for column in 1:columns
        pivot_row > rows && break
        candidate = findfirst(
            row -> !iszero(result[row, column]),
            pivot_row:rows,
        )
        isnothing(candidate) && continue
        selected = pivot_row + candidate - 1

        if selected != pivot_row
            result[pivot_row, :],
            result[selected, :] =
```

```

        copy(result[selected, :]),
        copy(result[pivot_row, :])
    end

    result[pivot_row, :] ./=
        result[pivot_row, column]

    for row in 1:rows
        row == pivot_row && continue
        factor = result[row, column]
        if !iszero(factor)
            result[row, :] -=
                factor .* result[pivot_row, :]
        end
    end
    pivot_row += 1
end

return result
end

```

A.2 Регистрация отдельных заданий

```

const TASKS = [
    "4.2.1" => run_4_2_1,
    "4.2.2" => run_4_2_2,
    "4.2.3" => run_4_2_3,
    "4.2.4" => run_4_2_4,
]

```

```
"4.2.5" => run_4_2_5,  
"4.2.6" => run_4_2_6,  
"4.4.1" => run_4_4_1,  
"4.4.2.1" => run_4_4_2_1,  
"4.4.2.2" => run_4_4_2_2,  
"4.4.3.1" => run_4_4_3_1,  
"4.4.3.2" => run_4_4_3_2,  
"4.4.3.3" => run_4_4_3_3,  
"4.4.4.1" => run_4_4_4_1,  
"4.4.4.2" => run_4_4_4_2,  
"4.4.4.3" => run_4_4_4_3,
```

```
]
```